2430598: PARALLEL COMPUTATION: RUST

B.Tech. II Year I Sem.	**L**	**T**	**P**	**C**
	0	**0**	**2**	**1**

Module I – Basics of Rust

1. **Hello, World!**
   - Basic Rust program structure using `fn main()`.
2. **Variable Declaration and Mutability**
   - Using `let`, `mut`, and type inference.
3. **Data Types Demo**
   - Use `i32`, `f64`, `char`, `bool`, etc., in a single program.
4. **Arithmetic and Logical Operators**
   - Calculator-style program showing arithmetic and logical operations.
5. **Using `cargo` for a Simple Project**
   - Create a simple project using `cargo new`, build, and run it.

Module II – Ownership and Control Flow

6. **Ownership Transfer**
   - Pass variables to functions and explore ownership and `move`.
7. **Borrowing and References**
   - Demonstrate `&T` and `&mut T` references and restrictions.
8. **Lifetimes Example**
   - Use lifetime annotations in a function returning references.
9. **If-Else Statement**
   - Even/odd checker or grade calculator.
10. **Looping Examples**
    - Use `for`, `while`, and `loop` to iterate over arrays or counters.
11. **Pattern Matching with `match`**
    - Match numbers to print weekdays or match enums.
12. **Nested Control Statements**
    - Combine loops and conditionals in one program (e.g., a simple number guessing game).

Module III – Functions and Structs

13. **Functions with Arguments and Return Values**
    - Create a function to calculate factorial or square of a number.

14. **Ownership in Functions**
    - Pass by value vs. reference in function parameters.
15. **Using Structs**
    - Define a `Rectangle` struct and calculate area.
16. **Tuple Structs and Field Init Shorthand**
    - Define a `Color(u8, u8, u8)` and use field shorthand for init.
17. **Enums and Pattern Matching**
    - Define an `enum` for `TrafficLight` and match its values.
18. **Option Enum Usage**
    - Safe division function returning `Option<f64>`.

Module IV – Project Management and Smart Pointers

19. **Error Handling with Result**
    - File reading or division with error handling using `Result`.
20. **Using Box, Rc, and RefCell**
    - Demonstrate smart pointers with a simple linked list or counter program.

Module- V – Object oriented Programming

21. **"Polymorphic Behavior with Trait Objects: A Speakable Animal Zoo"**
Covers: Traits, dynamic dispatch, trait objects (`&dyn Trait`)

22. **"Implementing Strategy Pattern with Traits for Payment Processing"**
Covers: Object-oriented design using traits and `Box<dyn Trait>` for dynamic strategy switching

23. **"Trait-Based Drawing Application with Heterogeneous UI Components"**
Covers: Object-oriented characteristics, trait objects, and allowing different drawable types in a single collection